

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Algorithm Design Challenge

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO1: Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 90 Mins

No. of Questions: 5

Annexure A Algorithm Design Challenge

Code of Conduct:

I K Sathwik Kumar (192425051) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K Sathwik Kumar

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

Answer all the Questions

Q.No	Question	Topics
1	Design an algorithm using Greedy Best-First Search to find a path from a source node to a goal node for the given Romania Map Problem graph. Clearly define the heuristic function, show the node expansion order, and justify why Greedy Best-First Search is suitable for this problem.	Informed Search, Greedy Best-First Search, Heuristic Functions
2	A delivery robot must travel from a warehouse to a customer through a weighted road network. Design an A Search algorithm to determine the optimal path. Calculate $g(n)$, $h(n)$, and $f(n)$ for each expanded node, and explain why A guarantees the optimal solution.	A* Search, Heuristic Functions
3	A university needs to prepare an examination timetable without scheduling conflicting subjects at the same time. Design the problem as a Constraint Satisfaction Problem (CSP) by identifying variables, domains, and constraints. Explain how MRV and Forward Checking improve the efficiency of the solution.	CSP, MRV, Forward Checking
4	Design the Minimax algorithm for the given game tree to determine the best move for the MAX player. Then apply Alpha-Beta Pruning to the same tree and indicate which nodes are pruned. Compare the number of nodes evaluated before and after pruning.	Game Playing, Minimax, Alpha-Beta Pruning
5	A warehouse robot operates in a dynamic environment where obstacles may appear unexpectedly. Design an intelligent search strategy using either Local Search or an Online Search Agent. Explain how the algorithm adapts to environmental changes and compare its performance with informed search algorithms.	Local Search, Optimization, Online Search Agents

Structure of the Answer – Algorithm Design Challenge

S.No	Sections	Expected Content
1	Problem Statement	Briefly describe the given problem in your own words.
2	Problem Analysis	Identify the initial state, goal state, assumptions, and constraints.
3	AI Technique Selection	Specify the most appropriate AI algorithm/search technique (e.g., Greedy Best-First Search, A*, CSP, Minimax, Local Search).
4	Justification of Algorithm	Explain why the selected algorithm is suitable compared to other search techniques.
5	State Space Representation	Represent the problem using a state-space diagram, search graph, game tree, or constraint graph, as applicable.
6	Variables, Domains, and Constraints (Applicable for CSP problems)	Identify variables, possible domains, and constraints. If not applicable, mention "Not Applicable".
7	Heuristic Function (Applicable for Informed Search)	Define the heuristic function $h(n)$ and explain whether it is admissible or appropriate for the problem. If not applicable, mention "Not Applicable".
8	Algorithm Design / Pseudocode	Write the step-by-step algorithm or pseudocode for solving the problem.
9	Flowchart / Algorithm Diagram	Draw a flowchart, search tree, game tree, or algorithm workflow illustrating the solution.
10	Working / Execution Trace	Demonstrate the execution of the algorithm with at least one example, showing intermediate steps such as node expansion, constraint propagation, or game-tree evaluation.
11	Complexity Analysis	Analyze the Time Complexity and Space Complexity of the proposed algorithm.
12	Advantages and Limitations	Discuss the strengths and limitations of the selected algorithm for the given problem.
13	Conclusion	Summarize the effectiveness of the designed algorithm and suggest possible improvements or alternative approaches.

Q1. Greedy Best-First Search – Romania Map

The original question asks you to design an algorithm using Greedy Best-First Search to find a path from a source node to a goal node in the given Romania Map graph, define the heuristic function, show the node expansion order, and justify why Greedy Best-First Search is suitable.

Below is the answer in the exact order of the assessment rubric, so you can write it directly.

1. Problem Statement

The objective is to find a path from a source city to a destination city in the Romania road map.

For this solution, we consider:

- Source node: Arad
- Goal node: Bucharest

Greedy Best-First Search uses a heuristic function to select the city that appears closest to the goal.

2. Problem Analysis

Initial State

The initial state is Arad.

Goal State

The goal state is Bucharest.

States

Each city in the Romania map represents a state.

Actions

An action represents travelling from one connected city to another through a road.

Constraints

1. The algorithm should avoid repeatedly expanding already visited nodes.
2. The next node is selected according to its heuristic value.
3. The search terminates when Bucharest is reached.

Expected Outcome

The algorithm should find a path from Arad to Bucharest.

3. AI Technique Selection

The selected AI technique is:

Greedy Best-First Search (GBFS).

Greedy Best-First Search is an informed search algorithm.

It uses the evaluation function:

[
 $f(n)=h(n)$
]

where:

- $(h(n))$ = estimated cost from node (n) to the goal.
- $(f(n))$ = priority of the node.

4. Justification of Algorithm

Greedy Best-First Search is suitable for the Romania Map problem because geographical information can be used as a heuristic.

The heuristic can be the straight-line distance from the current city to Bucharest.

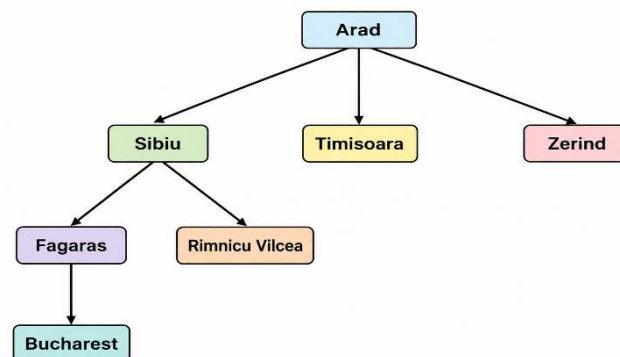
The algorithm focuses on cities that appear closer to the destination.

Therefore, it can reach the goal quickly compared with uninformed search methods.

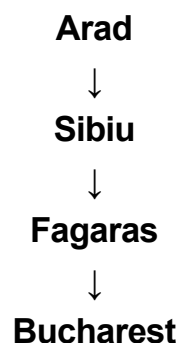
However, Greedy Best-First Search does not guarantee the shortest path because it does not consider the cost already travelled.

5. State-Space Representation

Use the following simplified representation:



The selected path is:



6. Heuristic Function

The heuristic function is:

[
 $h(n) = \text{Straight-line distance from } n \text{ to Bucharest}$
]

Example values:

City	Heuristic ($h(n)$)
Arad	366
Sibiu	253
Timisoara	329
Zerind	374

City	Heuristic ($h(n)$)
Fagaras	176
Rimnicu Vilcea	193
Bucharest	0

The smaller the heuristic value, the closer the city is estimated to be to Bucharest.

7. Algorithm / Pseudocode

Step 1: Start from Arad.

Step 2: Create an OPEN priority queue.

Step 3: Insert Arad into the OPEN queue.

Step 4: Calculate ($h(n)$) for Arad.

Step 5: Select the node having the smallest heuristic value.

Step 6: Check whether the selected node is Bucharest.

Step 7: If it is Bucharest, terminate the search.

Step 8: If it is not Bucharest, expand the node.

Step 9: Generate its unvisited neighbouring cities.

Step 10: Calculate the heuristic value for each neighbour.

Step 11: Insert the neighbours into the priority queue.

Step 12: Select the node with the smallest heuristic value.

Step 13: Repeat the process until Bucharest is reached.

Step 14: Return the path from Arad to Bucharest.

8. Working / Execution Trace

Step 1 – Expand Arad

Current node:

```
[
Arad
]
```

Heuristic:

```
[
h(Arad)=366
]
```

Neighbouring cities:

- Sibiu $\rightarrow$ ($h=253$)
- Timisoara $\rightarrow$ ($h=329$)
- Zerind $\rightarrow$ ($h=374$)

The smallest heuristic value is:

```
[
253
]
```

Therefore, Sibiu is selected.

Step 2 – Expand Sibiu

Current node:

```
[  
Sibiu  
]
```

Relevant neighbouring cities:

- Fagaras $\rightarrow$ ($h=176$)
- Rimnicu Vilcea $\rightarrow$ ($h=193$)

Compare:

```
[  
176 < 193  
]
```

Therefore, Fagaras is selected.

Step 3 – Expand Fagaras

Current node:

```
[  
Fagaras  
]
```

Neighbour:

```
[  
Bucharest  
]
```

Heuristic:

```
[  
h(Bucharest)=0  
]
```

Since Bucharest is the goal, the search terminates.

9. Node Expansion Order

The node expansion order is:

```
[  
\boxed{Arad\rightarrow Sibiu\rightarrow Fagaras\rightarrow Bucharest}  
]
```

Working Table

Step	Expanded Node	($h(n)$)	Selected Next
1	Arad	366	Sibiu
2	Sibiu	253	Fagaras
3	Fagaras	176	Bucharest
4	Bucharest	0	Goal

10. Final Path

Therefore, the path found by Greedy Best-First Search is:

[
\boxed{Arad\rightarrow Sibiu\rightarrow Fagaras\rightarrow Bucharest}
]

11. Complexity Analysis

Let:

- (b) = branching factor
- (m) = maximum depth of the search.

Time Complexity

[
 $O(b^m)$
]

Space Complexity

[
 $O(b^m)$
]

The algorithm may need to store many nodes in the OPEN and CLOSED structures.

12. Advantages

1. It uses heuristic knowledge.
2. It can reach the goal quickly.
3. It is suitable when an approximate route is acceptable.
4. It is simple to implement using a priority queue.
5. It is more directed than uninformed search.

13. Limitations

1. It does not guarantee the optimal path.
2. It depends on the quality of the heuristic.
3. A poor heuristic can lead to inefficient search.
4. It ignores the actual cost already travelled.
5. It can choose a path that looks promising but is actually more expensive.

14. Conclusion

Greedy Best-First Search is an informed search technique that uses a heuristic to guide the search toward the goal.

For the Romania Map problem, the algorithm selects the city with the smallest estimated distance to Bucharest.

The resulting path is:

[
\boxed{Arad\rightarrow Sibiu\rightarrow Fagaras\rightarrow Bucharest}
]

Therefore, Greedy Best-First Search is effective for quickly finding a route, although it does not guarantee the shortest or optimal route.

Q2. A* Search – Delivery Robot

1. Problem Statement

Line 1: A delivery robot must travel from a warehouse to a customer through a weighted road network.

Line 2: The robot needs to find the optimal path from the starting location to the destination.

Line 3: The A* Search algorithm is used to determine the optimal route.

Line 4: A* uses both the actual cost travelled and the estimated cost remaining to the goal.

2. Problem Analysis

Initial State

Line 1: The warehouse is represented by node A.

Goal State

Line 2: The customer is represented by node G.

States

Line 3: Each location in the road network represents a state.

Actions

Line 4: An action represents travelling from one connected location to another.

Path Cost

Line 5: Each road has a numerical weight representing its travelling cost.

Constraints

Line 6: The robot must travel through connected roads.

Line 7: The robot should find the minimum-cost route.

Line 8: The robot should avoid unnecessarily revisiting states.

Expected Outcome

Line 9: The expected outcome is the optimal path from the warehouse to the customer.

3. Example Weighted Road Network

For demonstrating the algorithm, consider:

More clearly:

$A \rightarrow B = 2$

$A \rightarrow C = 4$

$B \rightarrow D = 2$

$B \rightarrow G = 5$

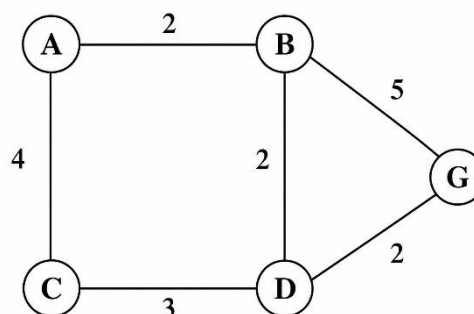
$C \rightarrow D = 3$

$D \rightarrow G = 2$

Here:

A = Warehouse

G = Customer



4. AI Technique Selection

*Selected Algorithm: A Search**

Line 1: A* is an informed search algorithm.

Line 2: It combines the actual cost from the starting node with the estimated cost to the goal.

Line 3: The evaluation function is:

Line 4: represents the actual cost from the start node to node .

Line 5: represents the estimated cost from node to the goal.

Line 6: represents the estimated total cost of a solution through node .

5. Heuristic Function

For this example, use the following heuristic values:

Node	
A	6
B	4
C	5
D	2
G	0

Line 1: The goal node has a heuristic value of zero.

Line 2: The heuristic estimates the remaining cost to reach the customer.

6. State-Space Representation

There is also a direct route:

$B \rightarrow G = 5$

The important possible routes are:

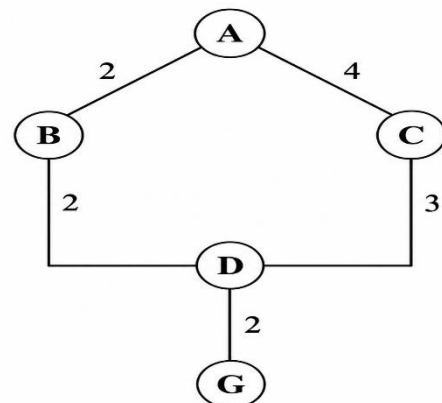
$A \rightarrow B \rightarrow G$

and

$A \rightarrow B \rightarrow D \rightarrow G$

and

$A \rightarrow C \rightarrow D \rightarrow G$



7. A* Algorithm / Pseudocode

Step 1: Start from the warehouse node A.

Step 2: Create an OPEN priority queue.

Step 3: Create a CLOSED set.

Step 4: Set:

Step 5: Calculate:

Step 6: Select the node with the smallest.

Step 7: Check whether the selected node is the goal.

Step 8: If it is the goal, reconstruct the path and return the solution.

Step 9: If it is not the goal, expand the node.

Step 10: Generate all neighbouring nodes.

Step 11: Calculate the tentative for every neighbour.

Step 12: Calculate.

Step 13: Add the nodes to the OPEN list.

Step 14: Select the node with the smallest.

Step 15: Repeat until the goal node is reached.

8. Execution Trace

Step 1 – Start at A

For A:

Therefore:

Expand A.

A has two successors:

B

Road cost:

Therefore:

Given:

Therefore:

C

Road cost:

Therefore:

Given:

Therefore:

9. OPEN List After Expanding A

Node			
B	2	4	6
C	4	5	9

The smallest value is:

Therefore:

is selected.

10. Step 2 – Expand B

B has two possible successors:

D

Road cost:

Therefore:

Given:

Therefore:

G

Road cost:

Therefore:

Given:

Therefore:

11. OPEN List:

Node			
D	4	2	6
G	7	0	7
C	4	5	9

The smallest value is:

Therefore:

is selected.

12. Step 3 – Expand D

D has a connection to G.

The cost is:

Therefore:

Since:

we get:

This is better than the previous value:

So the improved path to G is selected.

13. Step 4 – Goal Reached

The selected node is:

Since G represents the customer:

14. Final A* Calculation Table

Node			
A	0	6	6
B	2	4	6
C	4	5	9
D	4	2	6
G	6	0	6

15. Optimal Path

The final path is:

Total cost:

Therefore:

16. Why Does A* Guarantee the Optimal Solution?

Line 1: A* uses:

Line 2: represents the actual cost already travelled.

Line 3: estimates the remaining cost.

Line 4: If the heuristic is admissible, it never overestimates the actual remaining cost.

Therefore:

Line 5: Because the heuristic does not overestimate, A* can identify an optimal solution.

Line 6: For graph search, a consistent heuristic is normally used to ensure the optimality property cleanly.

17. Complexity Analysis

Let:

- = branching factor
- = depth of the optimal solution.

Time Complexity

Worst case:

Space Complexity

A* can require considerable memory because it stores nodes in its OPEN and CLOSED structures.

18. Advantages

1. A* can find an optimal path when the heuristic is admissible.
2. It considers both actual and estimated costs.
3. It is suitable for weighted road networks.
4. It is widely applicable to robot navigation and route planning.
5. A good heuristic can significantly reduce unnecessary exploration.

19. Limitations

1. A* can require large amounts of memory.
2. Its performance depends on the quality of the heuristic.
3. A poor heuristic can increase the number of nodes explored.
4. For very large search spaces, computation can become expensive.

20. Conclusion

A* Search is suitable for the delivery robot because the road network has different travelling costs and the robot needs an optimal route.

The algorithm evaluates each node using:

For the illustrative network, the optimal route is:

with total cost:

Thus, A* effectively combines the actual travelling cost and estimated remaining cost to find an optimal solution when an appropriate admissible heuristic is used.

1. Problem Statement

- A university needs to prepare an examination timetable without scheduling conflicting subjects at the same time.
- The problem can be formulated as a Constraint Satisfaction Problem (CSP).
- The objective is to assign a valid examination time slot to every subject while satisfying all constraints.

2. Problem Analysis

- Initial State: No subjects are assigned to any time slot.
- Goal State: All subjects are assigned valid examination slots.
- Variables: The subjects that need to be scheduled.
- Domains: The available examination time slots.
- Constraints: Subjects having common students must not be scheduled at the same time.

3. AI Technique Selection

- The selected technique is Constraint Satisfaction Problem (CSP).
- CSP is suitable because the problem consists of variables, domains, and constraints.

Two techniques are used to improve the search:

1. Minimum Remaining Values (MRV)
2. Forward Checking

4. Variables

Let the subjects be:

AI, DBMS, OS, Mathematics

Therefore:

$X = \{\text{AI, DBMS, OS, Mathematics}\}$

Each subject is treated as a variable.

5. Domains

Assume the available examination slots are:

T1 = Monday 9:00 AM

T2 = Monday 2:00 PM

T3 = Tuesday 9:00 AM

T4 = Tuesday 2:00 PM

Initially:

$\text{AI} = \{\text{T1, T2, T3, T4}\}$

$\text{DBMS} = \{\text{T1, T2, T3, T4}\}$

$\text{OS} = \{\text{T1, T2, T3, T4}\}$

$\text{Mathematics} = \{\text{T1, T2, T3, T4}\}$

6. Constraints

Assume the following subjects have common students:

AI $\neq$ DBMS

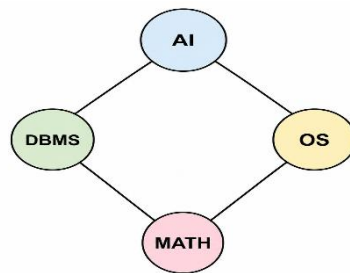
AI $\neq$ OS

DBMS $\neq$ Mathematics

OS $\neq$ Mathematics

This means conflicting subjects cannot be assigned the same examination slot.

7. Constraint Graph



8. Minimum Remaining Values (MRV)

MRV is a heuristic used to select the next variable in a CSP.

It selects the variable having the smallest number of available values.

For example, if:

AI = {T1, T2}

DBMS = {T1, T2, T3}

OS = {T1, T2, T3}

Mathematics = {T1, T2, T3, T4}

MRV selects AI because it has the smallest domain.

This helps identify difficult assignments early.

9. Forward Checking

Forward Checking removes an assigned value from the domains of neighbouring variables.

For example, if:

AI = T1

Since AI conflicts with DBMS and OS:

DBMS cannot use T1.

OS cannot use T1.

Therefore:

DBMS = {T2, T3, T4}

OS = {T2, T3, T4}

This reduces the number of invalid choices.

10. Algorithm

Step 1: Start with all subjects unassigned.

Step 2: Identify the available time slots.

Step 3: Select an unassigned subject using MRV.

Step 4: Select a valid time slot.

Step 5: Check whether the assignment satisfies the constraints.

Step 6: Assign the time slot.

Step 7: Apply Forward Checking.

Step 8: Remove the assigned slot from conflicting subjects.

Step 9: If any subject has no remaining slot, backtrack.

Step 10: Repeat until all subjects are assigned.

Step 11: Return the completed timetable.

11. Working / Execution Trace

Initially:

$AI = \{T1, T2, T3, T4\}$

$DBMS = \{T1, T2, T3, T4\}$

$OS = \{T1, T2, T3, T4\}$

$Mathematics = \{T1, T2, T3, T4\}$

Suppose MRV selects AI.

Assign:

$AI = T1$

Forward Checking removes T1 from DBMS and OS.

Therefore:

$DBMS = \{T2, T3, T4\}$

$OS = \{T2, T3, T4\}$

Now select DBMS.

Assign:

$DBMS = T2$

Forward Checking removes T2 from Mathematics.

Therefore:

$Mathematics = \{T1, T3, T4\}$

Now assign:

$OS = T2$

This is allowed because OS does not directly conflict with DBMS under the defined constraints.

Finally:

$Mathematics = T3$

All constraints are satisfied.

12. Final Timetable

Subject	Time Slot
AI	T1
DBMS	T2
OS	T2
Mathematics	T3

Therefore, the timetable is valid because no pair of conflicting subjects is scheduled at the same time.

13. Complexity Analysis

Let:

n = number of subjects

d = number of available time slots

The worst-case time complexity of CSP backtracking is:

$O(d^n)$

The space complexity is approximately:

$O(n + d)$

MRV and Forward Checking reduce the practical search space by selecting constrained variables and removing invalid values early.

14. Advantages

1. CSP provides a natural representation of timetable scheduling.
2. MRV selects the most constrained subject first.
3. Forward Checking removes invalid future assignments.
4. It reduces unnecessary backtracking.
5. Additional constraints can easily be added.

15. Limitations

1. The worst-case search can still be exponential.
2. A large number of subjects increases the search space.
3. Many complex constraints can make the problem difficult.
4. Poor variable selection can increase computation.

16. Conclusion

The examination timetable can be effectively represented as a Constraint Satisfaction Problem. Subjects are represented as variables, examination slots are represented as domains, and conflicts are represented as constraints. MRV improves the search by selecting the most constrained subject, while Forward Checking removes invalid future choices. Therefore, CSP with MRV and Forward Checking provides an efficient method for generating a conflict-free examination timetable. This follows the assessment's Q3 requirements for CSP, MRV, and Forward Checking. It also follows the required answer structure specified in the assessment.

Q4. Minimax and Alpha–Beta Pruning

1. Problem Statement

The objective is to find the best move for the MAX player using Minimax and improve the search using Alpha–Beta Pruning.

2. Problem Analysis

Initial State: Root node (MAX).

Goal State: Terminal nodes with utility values.

MAX tries to maximize the score.

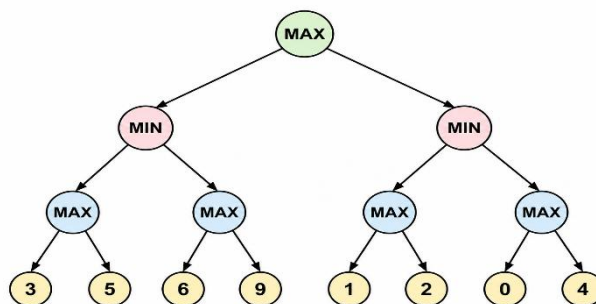
MIN tries to minimize the score.

3. Algorithm Selection

Minimax is suitable for two-player competitive games.

Alpha–Beta Pruning reduces unnecessary node evaluations without changing the final answer.

4. Game Tree



5. Minimax Calculation

Left subtree:

$$\text{MAX}(3,5) = 5$$

$$\text{MAX}(6,9) = 9$$

$$\text{MIN}(5,9) = 5$$

Right subtree:

$$\text{MAX}(1,2) = 2$$

$$\text{MAX}(0,4) = 4$$

$$\text{MIN}(2,4) = 2$$

Root:

$$\text{MAX}(5,2) = 5$$

Therefore, MAX selects the left branch.

6. Alpha–Beta Pruning

Alpha represents the best value for MAX.

Beta represents the best value for MIN.

Pruning occurs when:

[
 $\alpha \geq \beta$
]

After obtaining 5 from the left subtree, the value 9 can be pruned because it cannot change the MIN decision.

In the right subtree, after obtaining 2, the remaining nodes 0 and 4 can be pruned because:

[
 $\alpha=5 \geq \beta=2$
]

7. Pruned Nodes

Pruned nodes:

9, 0, 4

Without pruning:

8 terminal nodes evaluated.

With pruning:

5 terminal nodes evaluated.

Nodes saved:

[
 $8-5=3$
]

8. Complexity

Minimax time complexity:

[
 $O(b^d)$
]

Space complexity:

[
 $O(bd)$
]

Alpha–Beta best-case time complexity:

[
 $O(b^{\{d/2\}})$
]

9. Advantages

1. Minimax provides optimal decision-making against an optimal opponent.
2. Alpha–Beta reduces unnecessary calculations.
3. The final decision remains the same.

10. Limitations

1. Large game trees require high computation.
2. Performance depends on tree depth and branching factor.
3. Alpha–Beta works better with good move ordering.

11. Conclusion

Minimax selects the left branch with utility 5.

Alpha–Beta Pruning reduces the evaluated nodes from 8 to 5 while producing the same optimal decision.

Q5. Online Search Agent – Dynamic Warehouse Robot

1. Problem Statement

A warehouse robot must reach a destination in an environment where obstacles can appear unexpectedly.

The robot must detect changes and modify its path accordingly.

2. Problem Analysis

Initial State: Robot is at the warehouse starting position.

Goal State: Robot reaches the required destination.

Environment: The warehouse contains paths, obstacles, and dynamically changing conditions.

Constraint: The robot must avoid obstacles and reach the destination safely.

3. AI Technique Selection

The selected technique is Online Search Agent.

Online search is suitable because the robot does not have complete information about future obstacles.

The robot observes the environment while moving and updates its route when necessary.

4. Working Principle

The robot follows:

Observe → Update → Plan → Act

If an obstacle appears, the robot detects it, updates its map, and searches for a new path.

5. State Representation

S = Starting position

G = Goal

= Obstacle

S			#	
	#			
			#	
#				G

6. Algorithm

Step 1: Start from the initial position.

Step 2: Observe the surrounding environment.

Step 3: Update the internal map.

Step 4: Plan a path toward the goal.

Step 5: Check whether the planned path is blocked.

Step 6: If the path is clear, move to the next position.

Step 7: If an obstacle appears, stop the current plan.

Step 8: Update the map with the new obstacle.

Step 9: Calculate an alternative path.

Step 10: Continue moving toward the goal.

Step 11: Repeat until the goal is reached.

7. Execution Trace

Initially, the robot plans:

$S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$

Suppose a new obstacle appears at C.

The robot detects the obstacle.

The original path becomes invalid.

The robot updates its map.

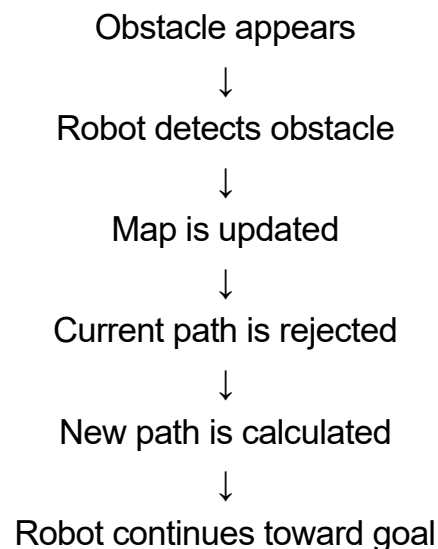
It then searches for an alternative route:

$S \rightarrow A \rightarrow B \rightarrow D \rightarrow E \rightarrow G$

The robot follows the new route and reaches the goal.

8. Adaptation

When the environment changes:



9. Comparison with Informed Search

Feature	Informed Search	Online Search
Environment	Mostly known	Partially unknown
Planning	Before movement	During movement
Dynamic obstacles	Less flexible	Highly adaptable
Replanning	Required when changes occur	Built into the approach
Suitable for dynamic robots	Moderate	High

10. Complexity Analysis

If A* is used for replanning, the worst-case time complexity is:

[
 $O(b^d)$
]

The space complexity is:

[
 $O(b^d)$
]

Repeated environmental changes may require multiple searches.

11. Advantages

1. Adapts to unexpected obstacles.
2. Works in partially unknown environments.
3. Continuously updates its knowledge.
4. Suitable for autonomous robots.
5. Can combine with A* for better route selection.

12. Limitations

1. Replanning can increase computation.
2. Sensor errors can affect decisions.
3. The initial route may not be optimal.
4. Continuous observation requires additional processing.

13. Conclusion

Online Search is suitable for a warehouse robot operating in a dynamic environment.

The robot continuously observes the environment, updates its knowledge, and changes its route when obstacles appear.

Therefore, Online Search provides a flexible approach for safe navigation in changing environments.

This directly addresses Q5 in the assessment, which asks for Local Search or an Online Search Agent and an explanation of how the algorithm adapts to environmental changes.

EVALUATION RUBRICS

Criteria	Weightage (%)	Excellent (90–100%)	Good (75–89%)	Average (60–74%)	Poor (<60%)
Problem Analysis and Understanding	15%	13–15% – Clearly identifies the problem, initial state, goal state, constraints, and expected outcome.	10–12% – Identifies most problem components with minor omissions.	7–9% – Demonstrates partial understanding with some missing elements.	0–6% – Incorrect or incomplete understanding of the problem.
AI Algorithm Selection & Justification	15%	13–15% – Selects the most appropriate AI algorithm/search technique with strong technical justification.	10–12% – Selects an appropriate algorithm with adequate justification.	7–9% – Selects a partially suitable algorithm with limited justification.	0–6% – Incorrect algorithm selection or no justification.
Algorithm Design / Pseudocode	25%	22–25% – Designs a complete, logical, efficient, and error-free algorithm/pseudocode.	17–21% – Algorithm is mostly correct with minor logical errors.	12–16% – Algorithm is partially complete with several logical errors.	0–11% – Algorithm is incomplete, incorrect, or difficult to understand.
Application of AI Concepts (Greedy, A*, CSP, Minimax, Alpha-Beta, etc.)	15%	13–15% – Correctly applies the relevant AI concepts and techniques throughout the solution.	10–12% – Applies AI concepts correctly with minor mistakes.	7–9% – Demonstrates limited application of AI concepts.	0–6% – Unable to apply the appropriate AI concepts or techniques.

Diagram / State Space / Search Tree Representation	10%	9–10% – Accurate, complete, and well-labelled diagram supporting the solution.	7–8% – Diagram is mostly correct with minor omissions.	5–6% – Diagram is partially complete or lacks clarity.	0–4% – Diagram is missing or incorrect.
Complexity Analysis and Solution Efficiency	10%	9–10% – Correctly analyzes time and space complexity and proposes an efficient solution.	7–8% – Complexity analysis is mostly correct with minor errors.	5–6% – Partial complexity analysis or limited discussion of efficiency.	0–4% – No meaningful complexity analysis or highly inefficient solution.
Originality, Critical Thinking, and Presentation	10%	9–10% – Demonstrates excellent originality, logical reasoning, organized presentation, and explores optimized/alternative solutions.	7–8% – Demonstrates good critical thinking with a clear presentation.	5–6% – Shows limited originality and acceptable presentation.	0–4% – Poor presentation with little or no critical thinking.